

Université de Rouen Normandie

Faculté des Sciences et Techniques
Département d'Informatique

Validation de syllogismes complexes à l'aide des diagrammes de Venn

Projet d'outil pour le calcul

Réalisé par

Kenza YOUSFI

Encadré par

Mr. Ludovic MIGNOT

Année universitaire 2025–2026

Table des matières

1	Introduction	2
2	Réponses aux questions du sujet	3
2.1	Justification de propriétés sur les diagrammes de Venn	3
2.2	Propriétés liées à la négation et à la conjonction de listes de diagrammes	4
3	Structures de données utilisées	5
3.1	Représentation des prédicats	5
3.2	Formules et syllogismes	5
3.3	Construction et manipulation des diagrammes	5
3.4	Modules Map et Set	6
3.5	Autres structures utilisées	6
4	Choix algorithmiques et récursivité	7
4.1	Méthode naïve	7
4.2	Méthode via négation	7
5	Difficultés rencontrées et solutions	8
6	Conclusion	9

Chapitre 1

Introduction

Les syllogismes constituent une classe particulière de raisonnements du calcul des prédicats, traditionnellement étudiée à l'aide des diagrammes de Venn. Dans ce cadre, les formules considérées mettent en jeu des prédicats unaires, quantifiés une seule fois, et portant sur une unique variable. Pour ce type de syllogismes, la validité peut souvent être déterminée à l'aide de la notion de compatibilité entre diagrammes de Venn.

Ce projet s'inscrit dans le prolongement de cette étude et s'intéresse à des *syllogismes complexes*, pour lesquels les prémisses et la conclusion peuvent être des combinaisons booléennes de formules quantifiées. Dans ce contexte, la notion classique de compatibilité simple s'avère insuffisante. L'objectif du projet est alors de définir et d'implanter des méthodes permettant de décider rigoureusement la validité de tels syllogismes, en s'appuyant sur les extensions complètes des diagrammes de Venn et sur une reformulation fondée sur la négation.

Chapitre 2

Réponses aux questions du sujet

2.1 Justification de propriétés sur les diagrammes de Venn

Propriété 1

Si deux diagrammes D et D' sont contradictoires, alors D n'est pas compatible avec D' .

Par définition, deux diagrammes sont contradictoires s'il existe une zone Z telle que $D(Z) = \text{Vide}$ et $D'(Z) = \text{NonVide}$, ou inversement. Or la compatibilité impose que deux diagrammes ne fixent jamais deux contraintes différentes sur une même zone lorsqu'elle est définie dans les deux. L'existence d'une telle zone Z exclut donc toute compatibilité entre D et D' .

Propriété 2

Si D et D' sont contradictoires, alors toute extension de D est en contradiction avec D' .

Si D et D' sont contradictoires, il existe une zone Z telle que $D(Z) \neq D'(Z)$. Par définition, une extension E de D conserve toutes les contraintes déjà présentes dans D et ne fait qu'en ajouter sur des zones auparavant indéfinies. On a donc nécessairement $E(Z) = D(Z)$, d'où $E(Z) \neq D'(Z)$. Toute extension de D reste ainsi contradictoire avec D' .

Propriété 3

Un diagramme D est compatible avec un diagramme D' si et seulement si D est une extension de D' .

Si D est une extension de D' , toutes les contraintes définies dans D' sont présentes dans D avec les mêmes valeurs, ce qui exclut toute contradiction : D est donc compatible avec D' . Réciproquement, si D est compatible avec D' , aucune contrainte de D' n'est contredite dans D . Le diagramme D peut alors être vu comme D' enrichi de contraintes supplémentaires, ce qui signifie que D est une extension de D' .

Propriété 4

Le nombre d'extensions (respectivement d'extensions complètes) d'un diagramme est 3^k (respectivement 2^k), où k est le nombre de zones non définies.

Pour chaque zone non définie, une extension autorise trois choix indépendants : laisser la zone indéfinie, la contraindre à *Vide* ou à *NonVide*. Le nombre total d'extensions est donc 3^k . Dans le cas des extensions complètes, aucune zone ne peut rester indéfinie : chaque zone offre alors seulement deux choix, ce qui conduit à 2^k extensions complètes.

Propriété 5

Si D est un diagramme complet, alors D et D' ne sont pas contradictoires si et seulement si D est compatible avec D' .

Lorsque D est complet, toutes les zones sont définies. L'absence de contradiction signifie alors qu'aucune zone définie dans D' ne reçoit une valeur différente dans D , ce qui correspond exactement à la définition de la compatibilité. Réciproquement, la compatibilité interdit toute zone portant deux contraintes opposées, ce qui garantit l'absence de contradiction.

2.2 Propriétés liées à la négation et à la conjonction de listes de diagrammes

On note $\text{Conj}(\ell_1, \ell_2)$ la conjonction de deux listes de diagrammes, c'est-à-dire l'ensemble des diagrammes obtenus en conjoignant un diagramme de ℓ_1 avec un diagramme de ℓ_2 lorsque la conjonction est cohérente. On note également $\neg d$ la négation d'un diagramme d (une liste de diagrammes unitaires, un par contrainte de d , avec la contrainte inversée) et $\neg \ell$ la négation d'une liste ℓ .

Propriété 1

La conjonction d'une liste de diagrammes ℓ avec la négation d'un diagramme d est vide si et seulement si tout diagramme de ℓ est compatible avec d .

($\Rightarrow$) Supposons $\text{Conj}(\ell, \neg d) = \emptyset$ et fixons $D \in \ell$. Supposons par l'absurde que D ne soit pas compatible avec d . Alors il existe une zone Z contrainte dans d telle que $D(Z) \neq d(Z)$. Par construction de $\neg d$, il existe un diagramme unitaire $N_Z \in \neg d$ qui impose précisément sur Z la contrainte opposée à $d(Z)$. On a alors $D(Z) = N_Z(Z)$, donc la conjonction $D \wedge N_Z$ est cohérente (au moins sur la zone conflictuelle), et elle produit un diagramme appartenant à $\text{Conj}(\ell, \neg d)$, contradiction. Donc tout $D \in \ell$ est compatible avec d .

($\Leftarrow$) Réciproquement, supposons que tout $D \in \ell$ soit compatible avec d . Prenons $D \in \ell$ et $N \in \neg d$. Par définition de $\neg d$, N impose sur une zone Z une contrainte opposée à celle de d , donc $N(Z) \neq d(Z)$. Or, comme D est compatible avec d , D ne peut pas imposer sur Z une contrainte différente de $d(Z)$; dès lors, D et N ne peuvent pas être simultanément satisfaits sur Z , et la conjonction $D \wedge N$ ne produit aucun diagramme cohérent. Ceci valant pour tout couple (D, N) , on obtient $\text{Conj}(\ell, \neg d) = \emptyset$.

Propriété 2

La conjonction d'une liste de diagrammes ℓ avec la négation d'une liste de diagrammes ℓ' ne contient que des extensions de diagrammes de ℓ en contradiction avec les diagrammes de ℓ' .

Par construction, un diagramme $E \in \text{Conj}(\ell, \neg \ell')$ est obtenu comme une conjonction $E = D \wedge N$ avec $D \in \ell$ et $N \in \neg \ell'$, et cette conjonction est cohérente. Comme la conjonction cumule les contraintes, E conserve toutes les contraintes de D et peut en ajouter : E est donc une extension de D .

De plus, $\neg \ell'$ est construit de sorte que tout $N \in \neg \ell'$ impose au moins une contrainte opposée à chaque diagramme $d' \in \ell'$ (il sélectionne, pour chacun, une zone contrainte et l'inverse). Ainsi, pour tout $d' \in \ell'$, il existe une zone Z telle que $N(Z) \neq d'(Z)$. Comme E contient les contraintes de N , on a aussi $E(Z) \neq d'(Z)$, donc E est en contradiction avec chaque $d' \in \ell'$. On conclut que tout élément de $\text{Conj}(\ell, \neg \ell')$ est une extension d'un diagramme de ℓ et qu'il est en contradiction avec les diagrammes de ℓ' .

Chapitre 3

Structures de données utilisées

3.1 Représentation des prédicats

Dans l'ensemble du projet, les prédicats unaires sont représentés par des chaînes de caractères. Ils apparaissent comme atomes dans les formules propositionnelles via le constructeur `Atome of string` du type `formule` (module `Formule`).

Plusieurs fonctions assurent la conversion des formules en représentations textuelles :

- `string_of_formule : formule -> string` convertit une formule propositionnelle en chaîne de caractères.
- `string_of_formule_log_prop_var : string -> formule -> string` convertit une formule propositionnelle en représentant chaque atome comme un prédicat unaire appliqué à une variable (par exemple $P(x)$).
- `string_of_formule_syllogisme : formule_syllogisme -> string` produit la représentation textuelle des formules quantifiées ($\forall x, \exists x$).

Ce choix de représentation simplifie l'égalité et la comparaison entre prédicats et permet leur utilisation directe comme éléments d'ensembles et comme clés de structures de données.

3.2 Formules et syllogismes

Les formules propositionnelles sont définies par le type algébrique `formule` (module `Formule`), avec les constantes `Top`, `Bot`, les atomes `Atome of string` et les connecteurs `Et`, `Ou`, `Imp` et `Non`.

Les formules utilisées dans les syllogismes sont modélisées par le type `formule_syllogisme` (module `Formule_Syllogisme`) : `PourTout of formule` et `IlExiste of formule`.

Enfin, afin de traiter des *syllogismes complexes*, on introduit le type `boolCombSyllogismes` permettant de construire des combinaisons booléennes arbitraires autour de formules de base, grâce aux constructeurs `Vrai`, `Faux`, `Base`, `Et`, `Ou` et `Non`.

3.3 Construction et manipulation des diagrammes

Un diagramme de Venn est représenté comme une application partielle associant à certaines zones une contrainte `fill` valant `Vide` ou `NonVide`. Les zones sont des ensembles de prédicats (type `Predicate_set.t`). Une zone absente de la structure correspond à une zone *indéfinie* (diagramme partiel).

La construction des diagrammes à partir de formules repose sur plusieurs étapes :

- **Extraction des prédicats** : Les fonctions `predicates_of_prop_formule`, `predicates_of_formule_syll` et `predicates_of_bool_comb` calculent l'ensemble des prédicats apparaissant dans une formule ou une combinaison booléenne.

- **Génération des zones** : La fonction `all_subsets` génère toutes les zones possibles (toutes les parties de l'ensemble de prédicats).
- **Satisfaction d'une zone** : La fonction `zone_satisfies` évalue une formule propositionnelle sur une zone : un atome P est vrai sur la zone si P appartient à l'ensemble représentant la zone.

La fonction `diag_from_formule` construit la liste des diagrammes associés à une formule quantifiée :

- pour $\forall x \varphi$, un unique diagramme est construit en marquant `Vide` toutes les zones qui ne satisfont pas φ ;
- pour $\exists x \varphi$, on construit un diagramme par zone satisfaisant φ , en imposant `NonVide` sur cette zone.

Les opérations logiques sur les listes de diagrammes sont implémentées par : `conj_diag` et `conj_diag_list` (conjonction cohérente), `disj_of_diag_list` (disjonction par union de listes) et `negate_diag` / `negate_diag_list` (négation). La fonction `diags_of_bool_comb` traduit récursivement une combinaison booléenne de syllogismes en une liste de diagrammes.

3.4 Modules Map et Set

Les diagrammes de Venn sont implémentés à l'aide de structures persistantes de la bibliothèque standard d'OCaml.

- **Set** : Le module `Predicate_set` est défini par `Set.Make(String)`. Il représente des ensembles de prédicats (chaînes de caractères) et sert à modéliser les *zones* des diagrammes de Venn : une zone est un ensemble de prédicats supposés vrais simultanément.
- **Map** : Le module `Diag` est défini par `Map.Make(Predicate_set)`. Un diagramme est alors une application partielle des zones vers une contrainte de remplissage. Plus précisément, on utilise le type `diagramme = fill Diag.t`, où `fill` vaut `Vide` ou `NonVide`.

Cette représentation permet de manipuler efficacement des diagrammes partiels : une zone non contrainte correspond à une zone absente de la Map.

3.5 Autres structures utilisées

Les listes sont largement utilisées pour représenter des collections finies, notamment des ensembles de prédicats, des listes de zones, ainsi que des listes de diagrammes de Venn. Elles servent également à représenter des combinaisons de diagrammes issues de la conjonction ou de la disjonction de contraintes, par exemple lors de la construction des diagrammes associés aux prémisses et aux conclusions.

Le projet repose sur plusieurs types algébriques pour modéliser les objets logiques manipulés. En particulier, les types `formule`, `formule_syllogisme` et `boolCombSyllogismes` permettent de représenter respectivement les formules propositionnelles, les formules quantifiées des syllogismes et les combinaisons booléennes de ces formules. Le type `fill`, défini par les constructeurs `Vide` et `NonVide`, permet quant à lui de représenter les contraintes associées aux zones des diagrammes de Venn.

Enfin, le type `option` est utilisé pour signaler explicitement les cas d'échec ou d'incompatibilité, notamment lors de la conjonction de diagrammes (`conj_diag`) ou de la recherche de témoins d'invalidité. Cette approche rend les cas d'erreur explicites et contribue à la sûreté des algorithmes implémentés.

Chapitre 4

Choix algorithmiques et récursivité

4.1 Méthode naïve

La méthode naïve suit directement la définition de la validité. À partir des diagrammes partiels décrivant les prémisses, on génère toutes leurs extensions complètes, puis on cherche un contre-exemple à la conclusion.

La fonction `complete_diags : diagramme -> string list -> diagramme list` construit toutes les extensions complètes d'un diagramme partiel en affectant `Vide` ou `NonVide` à chaque zone non définie. La contradiction entre deux diagrammes est testée par `is_contradiction` : deux diagrammes sont contradictoires s'ils imposent des contraintes différentes sur au moins une zone commune.

Le test principal `est_valid_premiss_conc` procède alors ainsi :

- construire l'ensemble `alpha` des prédicats apparaissant dans les deux combinaisons booléennes ;
- calculer les diagrammes des prémisses `dps` et de la conclusion `dcs` via `diags_of_bool_comb` ;
- si `dps` est vide (prémisses impossibles), la validité est vraie par vacuité ;
- sinon, rechercher une extension complète d'un diagramme de prémisses qui est en contradiction avec tous les diagrammes de la conclusion.

La fonction `temoins_invalidite_premisses_conc` permet en outre de renvoyer explicitement des contre-exemples sous forme de diagrammes complets.

4.2 Méthode via négation

La seconde méthode repose sur une reformulation équivalente : un syllogisme est valide si et seulement si la conjonction des prémisses avec la négation de la conclusion est impossible.

Dans l'implémentation, on calcule les diagrammes des prémisses `diags_b1` et ceux de la négation de la conclusion `diags_not_b2` en utilisant `diags_of_bool_comb alpha (Non b2)`. Le test de validité `est_valid_premiss_conc` consiste alors à vérifier que la conjonction `conj_diag_list diags_b1 diags_not_b2` est vide. Comme pour la méthode naïve, la validité est vraie par vacuité si les prémisses ne produisent aucun diagramme.

La fonction `temoins_invalidite_premisses_conc` fournit les témoins d'invalidité en renvoyant les diagrammes de la conjonction prémisses $\wedge \neg$ conclusion.

Chapitre 5

Difficultés rencontrées et solutions

La première difficulté rencontrée lors de la construction du projet a concerné la traduction des notions théoriques des diagrammes de Venn en structures de données exploitables en OCaml. En particulier, le passage de définitions abstraites (zones, contraintes, extensions) à une représentation concrète et fonctionnelle n'était pas immédiat.

La solution retenue a été de représenter les diagrammes de Venn comme des applications partielles associant à certaines zones une contrainte de vacuité ou de non-vacuité. Les zones non définies ont été volontairement laissées absentes de la structure, ce qui a permis de modéliser naturellement les diagrammes partiels et de simplifier la gestion des extensions.

La seconde difficulté a porté sur la génération correcte des extensions complètes des diagrammes de prémisses. Il était essentiel de garantir que toutes les extensions possibles soient prises en compte, tout en conservant les contraintes initiales du diagramme de départ. Cette étape s'est révélée délicate en raison du caractère combinatoire du problème.

Pour y répondre, une fonction dédiée, `complete_diags`, a été implémentée afin d'identifier explicitement les zones non définies d'un diagramme partiel et de générer récursivement toutes les extensions complètes possibles en leur affectant une contrainte de vacuité ou de non-vacuité. Bien que cette approche soit coûteuse en termes de complexité, elle assure la correction de la méthode naïve et constitue une base de référence fiable.

La troisième difficulté a concerné la mise en œuvre de la méthode via négation. La construction correcte de la négation d'un diagramme et la combinaison de listes de diagrammes par conjonction et disjonction devaient préserver une équivalence logique stricte avec la méthode naïve, sans recourir à la génération explicite des extensions complètes.

La solution adoptée a consisté à considérer la négation d'un diagramme comme une liste de diagrammes unitaires portant chacun une contrainte inversée. Cette reformulation a permis de ramener le test de validité à un simple test de vacuité d'une liste de diagrammes, simplifiant ainsi l'algorithme tout en garantissant sa correction.

Chapitre 6

Conclusion

Ce projet a permis d'étudier en profondeur la validité des syllogismes complexes à l'aide des diagrammes de Venn, en dépassant le cadre classique de la compatibilité simple. Les résultats obtenus mettent en évidence les limites de cette notion lorsqu'il s'agit de traiter des prémisses et des conclusions exprimées comme des combinaisons booléennes de formules quantifiées.

Deux approches complémentaires ont été développées et comparées. La méthode naïve, fondée sur la génération explicite des extensions complètes des diagrammes des prémisses, traduit directement la définition théorique de la validité, au prix d'une complexité exponentielle. La méthode via négation, quant à elle, repose sur une reformulation logique équivalente consistant à tester la vacuité de la conjonction des prémisses avec la négation de la conclusion. Cette seconde approche permet de décider plus efficacement la validité tout en garantissant une équivalence stricte avec la méthode naïve.

L'implémentation de ces méthodes a nécessité une modélisation rigoureuse des diagrammes de Venn, des zones et des contraintes associées, ainsi qu'une utilisation approfondie des structures persistantes et des types algébriques d'OCaml. Elle a également mis en évidence l'importance de définitions précises pour des notions telles que la contradiction, la compatibilité et la négation, afin d'assurer la correction des algorithmes.

Au-delà des résultats techniques, ce projet a constitué une expérience formatrice, renforçant ma compréhension de la programmation fonctionnelle, de la conception d'algorithmes et du raisonnement logique. Il représente une étape importante dans l'acquisition de compétences méthodologiques et théoriques, notamment dans le lien entre formalisme logique et implémentation informatique.